



MEETING ROOM RESERVATION SYSTEM PRESENTED

BY: MARY WARUGURU WANJIRU

REG. NO.: 20/05102

SUPERVISOR: JACOB OLOO

**SYSTEM DESIGN SPECIFICATION PRESENTED TO THE SCHOOL OF
TECHNOLOGY IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD OF A BACHELOR'S DEGREE IN INFORMATION AND
COMMUNICATION TECHNOLOGY AT KCA UNIVERSITY**

SUBMISSION DATE JANUARY 2025

ABSTRACT

This project proposes the development of a web-based Meeting Room Reservation Management System for Kenya Law to replace the existing manual process. The current system, reliant on verbal communication with a secretary, results in inefficiencies such as scheduling conflicts, lack of transparency, and delays. The proposed system will enhance efficiency, transparency, and accessibility by allowing employees to book rooms independently. It will include real-time availability tracking, automated notifications, and integration with Google Calendar.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Jacob Oloo, for his invaluable guidance and support throughout this project. I also extend my appreciation to my family, friends, and colleagues who provided encouragement and assistance during this research

DECLARATION

I, Mary Waruguru Wanjiru, declare that this project is my original work and has never been submitted to this or any other university for academic credit.

Supervisor _____ Signature: _____ Date: _____

Student _____ Signature: _____ Date: _____

Table of Content

ABSTRACT.....	ii
ACKNOWLEDGEMENT	iii
DECLARATION.....	iv
1.0 Introduction	1
1.1 Goals and Objectives	1
1.2 Statement of Scope	1
1.3 Software Context.....	1
1.4 Major Constraints	2
2.0 Data Design.....	3
2.1 Internal Software Data Structure.....	3
2.2 Global Data Structure	3
2.3 Temporary Data Structure.....	3
2.4 Database Description	4
3.0 Architectural and Component-Level Design.....	6
3.1 System Structure	6
3.1.1 Architecture Diagram	6
3.2 Description for Component	7
3.2.1 Processing Narrative for Component	7
3.2.2 Component Interface Description	7
3.2.3 Component Processing Detail	7
3.3 Dynamic Behavior for Component.....	9
3.3.1 Sequence Diagram for Reservation Use Case.....	9
3.4 Database Modelling.....	9
4.0 User Interface Design	11
4.1 Description of the User Interface.....	11
4.1.1 Screen Images	11
4.1.2 Objects and Actions.....	11
4.2 Interface Design Rules	12
4.3 Components Available	13
5.0 Restrictions, Limitations, and Constraints.....	14
6.0 Testing Issues	16
6.1 Classes of Tests.....	16

6.2 Expected Software Response	17
6.3 Performance Bounds	17
6.4 Identification of Critical Components	18
References.....	20

1.0 Introduction

This section provides an overview of the entire requirement document. This document describes all data, functional, and behavioral requirements for the Meeting Room Reservation Management System (MRRMS).

1.1 Goals and Objectives

1. Goal:

To develop a Meeting Room Reservation Management System that allows users to reserve, modify, and cancel meeting room bookings efficiently.

2. Objectives:

Provide a user-friendly interface for employees to book meeting rooms.

Enable administrators to manage meeting rooms and user accounts.

Ensure secure access to the system for authorized users only.

Generate reports for room utilization and reservations.

1.2 Statement of Scope

The MRRMS will allow users to:

- View available meeting rooms.

- Reserve a meeting room for a specific date and time.

- Modify or cancel existing reservations.

- Generate reports on room usage.

Major Inputs:

- User login credentials.

- Meeting room details (e.g., room name, capacity, location).

- Reservation details (e.g., date, time, duration).

Major Outputs:

- Reservation confirmation.

- Updated meeting room availability.

- Reports on room utilization.

- Ranking of Processing Functionality:

Essential: User authentication, room reservation, and cancellation.

Desirable: Email notifications, report generation, and integration with calendar systems.

Future: Mobile app integration, advanced analytics for room usage.

1.3 Software Context

The MRRMS will be used within an organization to streamline the process of booking meeting

rooms. It will integrate with existing email and calendar systems to send notifications and updates. The system will improve efficiency and reduce conflicts in room bookings.

1.4 Major Constraints

The system must be compatible with existing email and calendar systems.

The system must support multiple users accessing it simultaneously.

The system must be accessible via web browsers and mobile devices.

Development must be completed within one semester.

2.0 Data Design

The data design of the Meeting Room Reservation Management System (MRRMS) is a critical aspect of the system's architecture. It defines how data is structured, stored, and accessed within the system. This section provides a detailed description of the internal, global, and temporary data structures, as well as the database schema and its components.

2.1 Internal Software Data Structure

Internal data structures are used within the software to manage and process data during runtime. These structures are passed between components and are essential for the system's functionality.

1. Reservation Object: Represents a *meeting room reservation* made by a user.

Attributes:

```
reservationId: int
roomId: int
userId: int
startTime: datetime
endTime: datetime
status: varchar(20)
```

2. Represents a *meeting room* available for reservation.

Attributes:

```
roomId: int
roomName: varchar(50)
capacity: int
location: varchar(100)
```

2.2 Global Data Structure

Global data structures are accessible across multiple components of the system and are used to store shared data.

1. User Object: Represents a user of the system (e.g., employee, administrator).

Attributes:

```
userId: int
name: varchar(25)
email: varchar(255)
username: varchar(25)
role: varchar(10)
```

2.3 Temporary Data Structure

Temporary data structures are used for interim processing and are not persisted in the database.

Temporary Reservation Object: Used to hold reservation details temporarily during the reservation process.

Attributes:

tempReservationId: int
roomId: int
userId: int
startTime: datetime
endTime: datetime

2.4 Database Description

The MRRMS uses a relational database to store all persistent data. The database is designed to ensure data integrity, minimize redundancy, and support efficient querying.

1. Users Table: Stores information about system users.

Schema:

```
CREATE TABLE Users (  
    userId INT PRIMARY KEY,  
    name VARCHAR(25),  
    email VARCHAR(255),  
    username VARCHAR(25),  
    role VARCHAR(10)  
);
```

2. MeetingRooms Table: Stores details about meeting rooms available for reservation.

Schema:

```
CREATE TABLE MeetingRooms (  
    roomId INT PRIMARY KEY,  
    roomName VARCHAR(50),  
    capacity INT,  
    location VARCHAR(100)  
);
```

3. Reservations Table: Stores details about room reservations made by users.

Schema:

```
CREATE TABLE Reservations (  
    reservationId INT PRIMARY KEY,  
    roomId INT,
```

```
userId INT,  
startTime DATETIME,  
endTime DATETIME,  
status VARCHAR(20),  
FOREIGN KEY (roomId) REFERENCES MeetingRooms(roomId),  
FOREIGN KEY (userId) REFERENCES Users(userId)  
);
```

3.0 Architectural and Component-Level Design

This section provides a detailed description of the system architecture and the components that make up the Meeting Room Reservation Management System (MRRMS). The design focuses on ensuring scalability, modularity, and efficiency while meeting the system's functional and non-functional requirements.

3.1 System Structure

The MRRMS follows a two-tier architecture, which separates the system into two main layers: the Client Tier and the Server Tier. This architecture simplifies development, enhances scalability, and ensures clear separation of concerns.

Client Tier (Presentation Layer): Handles the user interface and user interactions.

Components:

- Web-based UI (HTML, CSS, JavaScript).

- Frameworks (e.g., React, Vue.js for dynamic UI).

Responsibilities:

- Displaying available rooms, reservation forms, and user dashboards.

- Collecting user input (e.g., reservation details).

- Sending user requests to the server tier and displaying responses.

Server Tier (Application and Data Layer): Handles application logic, data processing, and database interactions.

Components:

- Application Logic: Built using a backend framework (Spring Boot, or Node.js).

- Database: Relational database (e.g., MySQL,) for storing user, room, and reservation data.

Responsibilities:

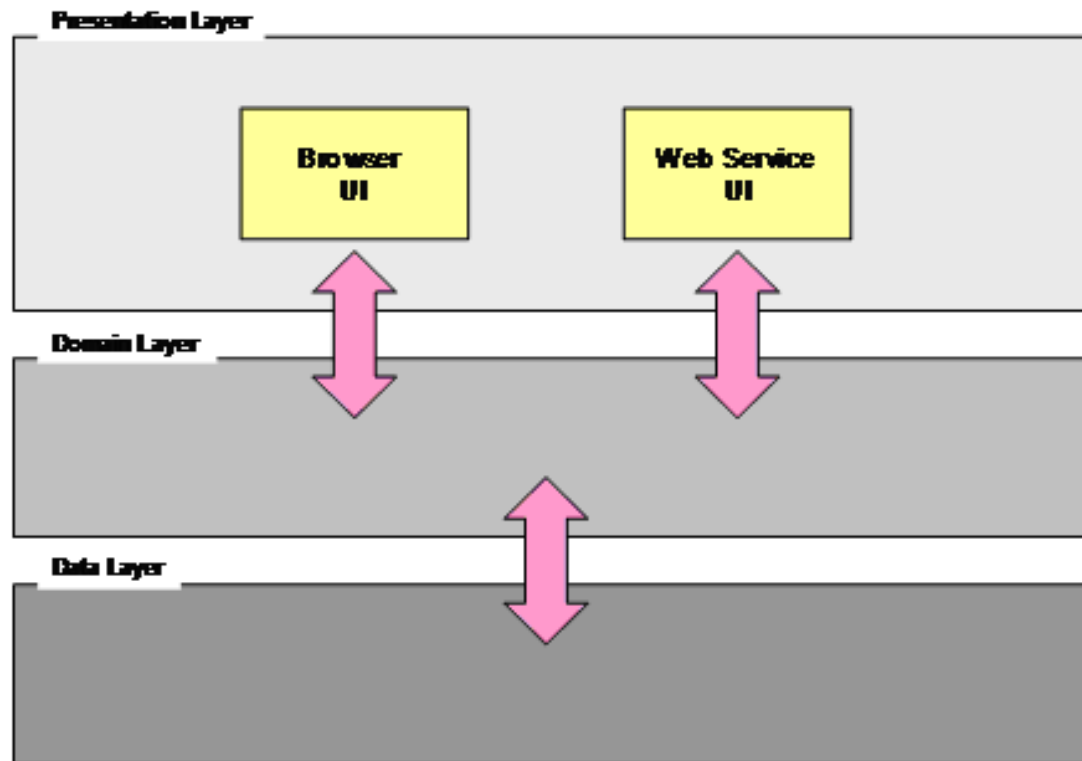
- Processing user requests (e.g., creating, modifying, or canceling reservations).

- Managing user authentication and authorization.

- Storing and retrieving data from the database.

3.1.1 Architecture Diagram

Below is a high-level UML component diagram illustrating the two-tier architecture:



3.2 Description for Component

The MRRMS is composed of several components, each responsible for specific functionality. Below is a detailed description of the Reservation Component, which is central to the system's operation.

3.2.1 Processing Narrative for Component

Reservation Component: Handles room reservations, modifications, and cancellations.

- Checking room availability for the requested date and time.

- Creating new reservations in the database.

- Modifying or canceling existing reservations.

- Ensuring that reservations do not overlap for the same room.

3.2.2 Component Interface Description

Input: Reservation details (e.g., roomId, userId, startTime, endTime).

Output: Reservation confirmation (e.g., "Reservation successful") or Error message (e.g., "Room not available").

3.2.3 Component Processing Detail

3.2.3.1 Design Class Hierarchy:

a) ReservationManager: Handles all reservation-related logic (e.g., creating, modifying, canceling reservations).

Methods:

reserveRoom(roomId, userId, startTime, endTime): Creates a new reservation.

modifyReservation(reservationId, newStartTime, newEndTime): Modifies an existing reservation.

cancelReservation(reservationId): Cancels an existing reservation.

b) UserManager: Handles user authentication and authorization.

Methods:

login(username, password): Authenticates users.

logout(userId): Logs out users.

3.2.3.2 Restrictions/Limitations:

Reservations cannot overlap for the same room.

Users cannot modify or cancel reservations that have already started.

3.2.3.3 Performance Issues:

The system must handle up to 100 concurrent users without significant performance degradation.

Database queries must execute within 500 milliseconds for 95% of operations.

3.2.3.4 Design Constraints:

The system must be accessible via web browsers.

The UI must be responsive and compatible with both desktop and mobile devices.

3.2.3.5 Processing Detail for Each Operation:

3.2.3.5.1 ReserveRoom Operation:

PSPEC (Process Specification):

Checks if the room is available for the requested date and time.

If available, creates a new reservation in the database.

Returns a confirmation message to the user.

If not available, returns an error message.

3.2.3.5.2 Algorithm:

IF room is available THEN

 CREATE reservation

 RETURN confirmation

ELSE

 RETURN error message

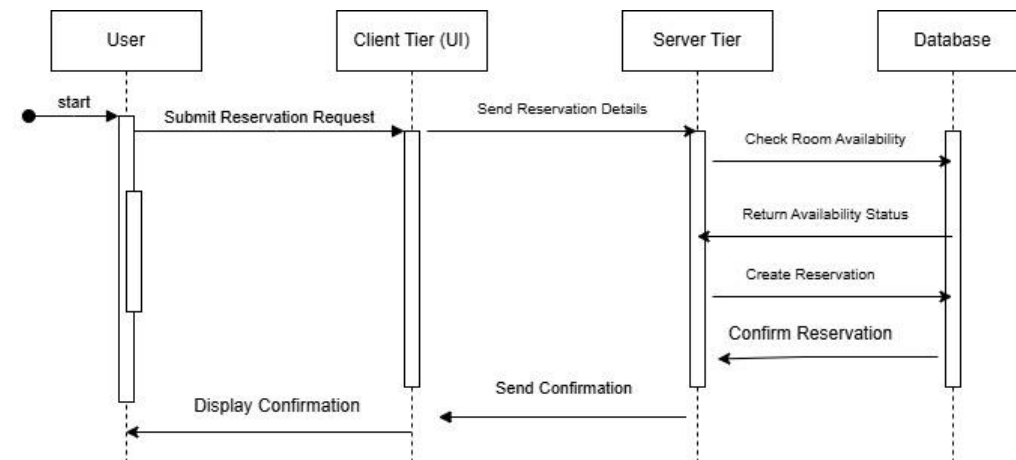
ENDIF

3.3 Dynamic Behavior for Component

The dynamic behavior of the system is illustrated using sequence diagrams, which show the interaction between components during specific use cases.

3.3.1 Sequence Diagram for Reservation Use Case

Below is a sequence diagram for the "Reserve Room" use case:



3.4 Database Modelling

The database is a critical component of the MRRMS, storing all data related to users, rooms, and reservations. The database design follows best practices for normalization and efficiency.

1. Data Normalization:

Tables are normalized to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity.

Example:

Users Table: Stores user information.

MeetingRooms Table: Stores room details.

Reservations Table: Stores reservation details with foreign keys linking to Users and MeetingRooms.

2. Entity-Relationship Model (ER Diagram):

The ER diagram illustrates the relationships between entities (e.g., Users, MeetingRooms, Reservations).

Relationships:

One-to-Many: A user can have multiple reservations, but each reservation belongs to one user.

One-to-Many: A room can have multiple reservations, but each reservation is for one room.

3. Database Schema:

See Section 2.4 for the detailed schema.

4. Data Dictionary:

Users:

- userId: Unique identifier for each user.
- name: User's full name.
- email: User's email address.
- username: User's login username.
- role: User's role (e.g., employee, administrator).

MeetingRooms:

- roomId: Unique identifier for each room.
- roomName: Name of the room.
- capacity: Maximum capacity of the room.
- location: Physical location of the room.

Reservations:

- reservationId: Unique identifier for each reservation.
- roomId: Foreign key linking to the MeetingRooms table.
- userId: Foreign key linking to the Users table.
- startTime: Start time of the reservation.
- endTime: End time of the reservation.
- status: Status of the reservation (e.g., confirmed, canceled).

4.0 User Interface Design

The user interface (UI) is a critical component of the Meeting Room Reservation Management System (MRRMS) as it directly impacts user experience and system usability. A well-designed UI ensures that users can interact with the system intuitively and efficiently. This section provides a detailed description of the UI, including screen layouts, objects, actions, design rules, and available components.

4.1 Description of the User Interface

The MRRMS UI is designed to be simple, intuitive, and user-friendly. It consists of several screens, each serving a specific purpose in the reservation process. Below is a detailed description of the key screens and their components:

4.1.1 Screen Images

Login Screen: Authenticates users and grants access to the system based on their role (e.g., employee, administrator).

Room Reservation Screen: Allows users to view available rooms and make reservations.

Dashboard screen: Shows a list of available rooms with details (e.g., room name, capacity, location).

Reservation Management Screen: Allows users to view, modify, or cancel their existing reservations.

Buttons: "Modify": To edit the selected reservation (e.g., change room, date, or time), "Cancel": To delete the selected reservation.

Search Bar: Allows users to search for specific reservations by date or room.

4.1.2 Objects and Actions

1. Buttons:

Reserve: Initiates the room reservation process.

Modify: Allows users to edit an existing reservation.

Cancel: Deletes a reservation or clears the current form.

2. Dropdowns:

Room Selection: Displays a list of available rooms for the selected date and time.

Time Selection: Allows users to choose the start and end times for their reservation.

3. Icons:

Calendar Icon: Opens a date picker for selecting the reservation date.

Search Icon: Used in the search bar to find specific reservations.

4. Pop-ups:

Confirmation Pop-up: Appears after a user reserves, modifies, or cancels a room to confirm the action.

Error Pop-up: Displays error messages (e.g., "Room not available") to inform users of issues.

4.2 Interface Design Rules

The MRRMS UI adheres to the eight golden rules of interface design to ensure a seamless and intuitive user experience:

Strive for Consistency:

Consistent design elements (e.g., button styles, fonts, colors) are used across all screens.

Keyboard shortcuts (e.g., "Ctrl + S" to save) are provided for power users.

Frequently used actions (e.g., reserving a room) are prominently displayed and easily accessible.

Offer Informative Feedback:

Users receive immediate feedback for their actions (e.g., "Reservation successful" pop-up).

Error messages are clear and provide guidance on how to resolve issues (e.g., "Room not available. Please select a different time.").

Design Dialogs to Yield Closure:

Actions (e.g., reserving, modifying, canceling) are completed within a single dialog or screen.

Confirmation pop-ups indicate the successful completion of an action.

Offer Error Prevention and Simple Error Handling:

The system prevents errors by validating user input (e.g., ensuring the end time is after the start time).

Permit Easy Reversal of Actions:

Users can easily undo actions (e.g., cancel a reservation) without navigating through multiple screens.

A "Cancel" button is provided on all forms to allow users to discard changes.

Support Internal Locus of Control:

Users feel in control of the system, with clear navigation and predictable outcomes for their actions.

Reduce Short-Term Memory Load:

Important information (e.g., selected room, reservation details) is displayed prominently on the screen.

Users are not required to remember information from one screen to another.

4.3 Components Available

The MRRMS UI is built using standard GUI components to ensure familiarity and ease of use. The following components are available for implementation:

Buttons: Used for actions such as reserving, modifying, or canceling a room. Examples: "Reserve," "Modify," "Cancel."

Dropdowns: Used for selecting options from a list (e.g., room selection, time selection).

Text Fields: Used for inputting text (e.g., username, password).

Icons: Used to represent actions or provide visual cues. Examples: Calendar icon for date selection, search icon for the search bar.

Pop-ups: Used to display messages or confirmations. Examples: "Reservation successful," "Room not available."

Search Bar: Allows users to search for specific reservations or rooms. Example: "Search by date or room name."

5.0 Restrictions, Limitations, and Constraints

The development and implementation of the Meeting Room Reservation Management System (MRRMS) are subject to several restrictions, limitations, and constraints. These factors must be carefully considered during the design and development phases to ensure the system meets its objectives while adhering to practical and organizational boundaries. Below is an expanded explanation of the key constraints:

- **Development Timeline:** The system must be developed, tested, and deployed within the duration of one academic semester (approximately 3 months).
- **Integration with Existing Email and Calendar Systems:** The MRRMS must integrate with the organization's existing email and calendar systems (e.g., Google Calendar).
- **Support for Up to 100 Concurrent Users:** The system must support up to 100 users without significant performance degradation.
- **Budget Constraints:** The project may have a limited budget for software licenses, hardware, or third-party services (e.g., cloud hosting, API subscriptions). Open-source tools and technologies will be prioritized to minimize costs.
- **User Training and Adoption:** The system must be intuitive and user-friendly to minimize the need for extensive training.
- **Security and Compliance:** The system must comply with the organization's security policies and data protection regulations.
- **Security measures,** such as encryption, secure authentication, and regular security audits, must be implemented.

Mitigation Strategies

- To address these constraints, the following mitigation strategies will be employed:
- **Agile Development:** Use an agile development methodology to deliver the system in iterative phases, ensuring that core functionality is completed first.
- **Prioritization:** Focus on essential features (e.g., room reservation, user authentication) and defer desirable or future features to later phases.
- **Scalable Architecture:** Design the system with scalability in mind, using technologies and practices that support high concurrency and performance.
- **Continuous Communication:** Maintain open communication with stakeholders (e.g., clients, users) to ensure the system aligns with their needs and constraints.

6.0 Testing Issues

Testing is a critical phase in the software development lifecycle to ensure the Meeting Room Reservation Management System (MRRMS) meets its functional, performance, and security requirements. This section outlines the testing strategy, expected outcomes, performance benchmarks, and identification of critical components that require rigorous testing.

6.1 Classes of Tests

The testing process will include both black-box and white-box testing methodologies to ensure comprehensive coverage of the system's functionality and internal logic.

Black-Box Testing:

Purpose: To validate the system's functionality from the user's perspective without knowledge of the internal code structure. Test Cases:

Reservation Functionality:

- Verify that users can successfully reserve a meeting room.

- Ensure that overlapping reservations are prevented.

Modification Functionality:

- Verify that users can modify existing reservations (e.g., change time or room).

Cancellation Functionality: Verify that users can cancel reservations and that the room becomes available for others.

Room Availability Updates: Ensure that room availability is updated in real-time after reservations, modifications, or cancellations.

White-Box Testing:

Purpose: To test the internal logic, structure, and code of individual components. Test Cases:

ReservationManager Component:

- Verify that the logic for checking room availability works correctly.

- Test edge cases, such as reservations at midnight or for very short durations.

User Authentication Component:

- Verify that user login and role-based access control function as intended.

- Test for security vulnerabilities, such as SQL injection or unauthorized access.

Database Operations:

Verify that database queries (e.g., INSERT, UPDATE, DELETE) execute correctly and efficiently.

6.2 Expected Software Response

The system is expected to respond accurately and efficiently to user actions. Specific expected responses include:

a) Reservation Creation:

When a user reserves a room, the system should:

Confirm the reservation with a unique reservation ID.

Update the room availability status in real-time.

Send a confirmation email to the user (if email integration is implemented).

b) Reservation Modification:

When a user modifies a reservation, the system should:

Update the reservation details in the database.

Reflect the changes in the room availability status.

Notify the user of the successful modification.

c) Reservation Cancellation:

When a user cancels a reservation, the system should:

Remove the reservation from the database.

Update the room availability status to "available."

Notify the user of the successful cancellation.

d) Room Availability Updates:

The system should ensure that room availability is updated in real-time and accurately reflects the current state of reservations.

6.3 Performance Bounds

The system must meet specific performance benchmarks to ensure a smooth user experience:

The system must respond within 2 seconds for 90% of user requests.

Examples of user requests include:

Loading the list of available rooms.

Creating, modifying, or canceling a reservation.

Logging in or logging out of the system.

Concurrency Handling:

The system must support up to 100 concurrent users without significant performance degradation.

Database Performance:

Database queries should execute within 500 milliseconds for 95% of operations.

Indexes will be used to optimize query performance for frequently accessed tables (e.g., Reservations, MeetingRooms).

6.4 Identification of Critical Components

Certain components of the system are critical to its functionality and require rigorous testing to ensure reliability and security.

Reservation Component:

Criticality: High

Why: This component is responsible for managing room reservations, modifications, and cancellations. Any failure in this component could lead to double bookings, incorrect availability status, or user dissatisfaction.

Validate the logic for checking room availability.

Ensure that database updates are atomic and consistent.

User Authentication Component:

Criticality: High

Why: This component ensures that only authorized users can access the system. A failure in this component could lead to security breaches or unauthorized access.

Testing Focus:

Verify that user login and role-based access control function correctly.

Test for security vulnerabilities, such as SQL injection, brute force attacks, or session hijacking.

Ensure that passwords are stored securely using hashing algorithms.

Database Component:

Criticality: High

Reason: The database is the backbone of the system, storing all critical data (e.g., users, rooms, reservations). Any failure in this component could lead to data loss or corruption.

Verify that database queries execute correctly and efficiently.

Test for data integrity and consistency.

Ensure that backups are performed regularly to prevent data loss.

Notification Component:

Criticality: Medium

Reason: This component is responsible for sending notifications (e.g., reservation confirmations, modifications) to users. While not critical to core functionality, it enhances user experience.

Verify that notifications are sent correctly and promptly.

Test for edge cases, such as invalid email addresses or network failures.

References

Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach.

Scott W. Ambler (2009). UML 2 Class Diagrams.

IEEE Std 1016-2009. IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.